

Peak 11 Media

Domain & Go-Live SOP — Technical Delivery Runbook

The exact, copy-pasteable process for hosting a client site free on GitHub Pages, pointing the client's own domain at it, and taking the site live cleanly. Updated June 2026.

0. The Hosting Model in One Paragraph

Every Peak 11 site is built as plain static files (HTML/CSS/images) and hosted **free, forever, on GitHub Pages** under the **consciousmirage** GitHub account. Previews live as subpaths under one shared repo so prospects only ever see their own link. When a deal closes, the client gets a **real domain we buy for them** (~\$10–12/yr, billed into the \$900 package) and we point it at their site. There are no monthly hosting bills — the recurring Care Plan covers updates, not servers.

There is exactly ONE paid step in this whole runbook.

Buying the domain (Section 1) is the only money-spend. **Defer it until the deal is closed and paid** — never pre-buy domains on spec. Everything else (hosting, HTTPS, deploys) is free.

1. Buying the Client's Domain (the one paid step)

Buy the client's domain under the **peak11media** registrar account so all client domains renew in one place we control. Pick the cheapest registrar with free WHOIS privacy and no surprise renewal jumps.

Registrar comparison (typical .com pricing, 2026)

Registrar	1st-yr .com	Renewal	WHOIS privacy	Verdict
Porkbun	~\$9–11	~\$11	Free	BEST — cheapest, clean UI, free privacy, easy DNS
Cloudflare	~\$10.44 (at-cost)	~\$10.44 (at-cost)	Free	Great if also using CF DNS; no markup, no upsells
Namecheap	~\$6–10 promo	~\$15+	Free 1st yr	OK, but renewal jumps — watch the price

Default choice: Porkbun. Cheapest flat pricing, free WHOIS privacy, free SSL, and the DNS panel is the simplest to operate. Use Cloudflare only when the client also wants Cloudflare's DNS/CDN.

Purchase steps (Porkbun)

- Log in to **porkbun.com** under the peak11media account (create it once; store creds in the password vault).
- Search the exact-match domain — prefer **businessname.com**; if taken, try **.com** variants (getX, Xutah, theX) before settling for .net/.co.
- Add to cart. Confirm **WHOIS privacy is ON** (free) and **auto-renew is ON**.
- Pay. This is the single billed step — log the ~\$10–12 against that client's \$900 in the CRM.
- Done. DNS is managed in Porkbun's **Details > DNS** for that domain (Section 2).

Deferred until a deal closes.

Do NOT buy any domain during sales/preview. The preview lives free on GitHub Pages. Only after the client says yes and pays do you buy the domain and run Section 2.

2. Pointing the Domain at GitHub Pages (DNS)

GitHub Pages serves an apex domain (e.g. **avalancheroofing.com**) via four **A** records, and the **www** subdomain via one **CNAME**. Set all five records in the registrar's DNS panel.

2a. The exact DNS records (copy-paste)

In Porkbun > Domain > DNS, delete any default parking records, then add:

Type	Host / Name	Value	TTL
A	@	185.199.108.153	600
A	@	185.199.109.153	600
A	@	185.199.110.153	600
A	@	185.199.111.153	600
CNAME	www	consciousmirage.github.io	600

Notes: the four A-record IPs are GitHub's official Pages IPs and are the same for every client. The CNAME value is **<owner>.github.io** — use the org name (e.g. **peak11-clients.github.io**) once sites move to the dedicated org. Some panels use a blank host instead of @ for the apex.

2b. Add the custom domain in the repo + the CNAME file

The repo needs a **CNAME** file at the served root containing just the domain, and the Pages config must list the domain. The helper script **set-custom-domain.sh** does both via gh api, but here is what it does manually:

```
# 1) Set the custom domain on the Pages site (also writes config)
gh api repos/<owner>/<repo>/pages -X PUT -f cname=avalancheroofing.com

# 2) Ensure a CNAME file exists at the served root containing the domain
# (the deploy helper commits this for you):
echo 'avalancheroofing.com' > CNAME # one line, no http://, no slash
```

2c. Enable enforced HTTPS

After the domain resolves, GitHub auto-provisions a free Let's Encrypt certificate (takes a few minutes). Once the cert is ready, turn on enforced HTTPS so the site is always served over https://:

```
# Retry until the cert finishes provisioning (this can take a few minutes)
gh api repos/<owner>/<repo>/pages -X PUT -F https_enforced=true

# In the GitHub UI: Settings > Pages > check "Enforce HTTPS"
```

2d. Verify propagation

```
# DNS – should print the four GitHub IPs:
dig +short avalancheroofing.com

# www – should print <owner>.github.io:
dig +short www.avalancheroofing.com

# Live + HTTPS – should print HTTP/2 200:
curl -sI https://avalancheroofing.com | head -1

# Pages status from the API:
gh api repos/<owner>/<repo>/pages --jq '{cname,https_enforced,status,html_url}'
```

Propagation timing.

Porkbun/Cloudflare DNS usually goes live in minutes; worst case is 24h. If GitHub says "Domain's DNS record could not be retrieved," the records haven't propagated yet — wait and re-check with dig. Don't change records repeatedly; that resets the timer.

3. Hosting Structure for Many Clients (the decision)

Three options were on the table. Here is the trade-off and the chosen model for a non-technical operator whose AI manages everything:

Model	Pros	Cons
One repo, subpaths (current: forge-demos)	Dead simple; one place; one deploy target; great for previews on OUR domain.	A repo can carry only ONE custom domain. Can't give each client their own domain from here.
One repo per client	Each client gets their OWN custom domain + HTTPS; clean isolation; easy handoff/transfer.	More repos to track (fine — the AI manages them via gh api).
GitHub Org (peak11-clients)	Central owner not tied to a personal login; clean org page; scales to many client repos; transferable.	One-time org setup; needs admin:org scope to create via API (or create in the UI).

DECISION: a hybrid that maps to the sales stage.

(1) Previews stay in ONE shared repo with subpaths — keep using **consciousmirage/forge-demos** for every demo/preview; the prospect only sees their own link. **(2) A PAID client who goes live on their OWN domain gets their OWN repo**, because a Pages repo can host only one custom domain. **(3) Going forward, create a dedicated GitHub org, peak11-clients, and put each paid client's repo there** (e.g. **peak11-clients/avalanche-roofing**). The org keeps client sites off the personal consciousmirage account, gives a clean central owner, and makes ownership transfer trivial if a client ever leaves. The AI manages all of it via gh api — repo count is not a burden.

Why an org over the personal account

- **Separation** — client production sites live under a business-named owner, not Nate's personal alias.
- **Transferability** — if a client cancels or wants to self-host, transfer the single repo cleanly.
- **Scale & clarity** — every paid client is one repo in one org; the CRM maps client → repo → domain 1:1.

- **No cost** — a GitHub org and unlimited public Pages repos are free.

One-time org setup

```
# Create the org in the GitHub UI (github.com/organizations/new, Free plan),
# OR via API if the token has admin:org scope:
#   (current token has repo + read:org; add admin:org to create via API)

# Create a paid client's repo under the org and deploy to ROOT:
gh repo create peak11-clients/avalanche-roofing --public \
  --description 'Avalanche Roofing - Peak 11 Media'

REPO=peak11-clients/avalanche-roofing \
  ./deploy-client.sh ../clients/avalanche-roofing/v3-preview '' 'Initial go-live'
```

Hardening already applied: .nojekyll.

A **.nojekyll** file was added to the hosting repo. GitHub Pages runs Jekyll by default, which silently drops any file or folder starting with an underscore (e.g. an **_assets** image folder) — that would break a client site with no error. The **.nojekyll** file disables Jekyll so every file is served verbatim. Add it to every client repo (the deploy helper can include it).

4. Go-Live Checklist

Run pre-launch QA on the preview URL BEFORE pointing the domain. Then launch, verify, and tell the client.

4a. Pre-launch QA (on the preview URL)

- **Mobile** — open on a phone (or DevTools device mode). Nothing overflows; text readable; tap targets big enough; menu works.
- **All links work** — click every nav item, button, and footer link; no 404s, no dead anchors.
- **tel: links** — phone numbers are `<a href="tel:+1801...">` and dial when tapped on mobile.
- **Email / forms** — mailto: or contact form submits and reaches the right inbox; test one real submission.
- **Images** — all load (no broken icons), correctly sized, not stretched; hero looks sharp on retina.
- **Page title** — browser tab shows "Business Name | City Trade" (the `<title>` tag), not "Document".
- **Favicon** — a favicon is set and shows in the tab.
- **SEO basics** — `<title>` + meta description present; one `<h1>`; image alt text; address/phone (NAP) consistent.
- **Content accuracy** — business name, phone, address, hours, services all correct (owner-confirmed).
- **Spelling** — proofread headlines and body; no lorem ipsum or placeholder text left.
- **HTTPS** — the preview already loads over https with no mixed-content warnings.

4b. Launch

- Buy the domain (Section 1) — the one paid step. Log the cost in the CRM.
- Create the client repo under **peak11-clients** and deploy the approved site to root (Section 3).
- Set the five DNS records at the registrar (Section 2a).
- Attach the custom domain + CNAME file and enforce HTTPS (Section 2b–2c / set-custom-domain.sh).

- Wait for DNS propagation and the Let's Encrypt cert (minutes, up to 24h worst case).

4c. Post-launch verification (on the LIVE domain)

- **https://thedomain.com loads** and shows the right site (curl -sI returns 200).
- **www redirects** — www.thedomain.com reaches the same site.
- **HTTPS enforced** — http:// auto-upgrades to https://; padlock shows; no cert warning.
- **Re-run mobile + link + tel: checks** on the live domain, not just the preview.
- **Google can see it** — submit the domain in Google Search Console (optional but recommended).
- **Screenshot** the live homepage for the CRM / portfolio.

4d. Tell the client they're live

Short, branded handoff. Send the live link, confirm it's theirs, and restate the Care Plan:

Go-live message (template)

"You're live — **https://avalancheroofing.com** is up. It's fully yours: mobile-friendly, secure (that padlock means HTTPS), and built to bring in calls. Save the link and share it anywhere you list your business. Any time you need a change — new photos, updated hours, a new service — just text me and I'll handle it. Welcome aboard."

Then: mark the client **LIVE** in the CRM with their domain + repo, and if they're on the \$149/mo Care Plan, confirm billing is active.

5. Quick Reference — the whole go-live in commands

```
cd /Users/.../Business/PrimoSites/deploy

# 1. Deploy approved site to the client's own repo root
gh repo create peak11-clients/avalanche-roofing --public
REPO=peak11-clients/avalanche-roofing \
  ./deploy-client.sh ../clients/avalanche-roofing/v3-preview '' 'Go-live'

# 2. (Buy domain at Porkbun — the one paid step — set 4 A + 1 CNAME)

# 3. Attach domain + CNAME file + enforce HTTPS
REPO=peak11-clients/avalanche-roofing \
  ./set-custom-domain.sh avalancheroofing.com

# 4. Verify
dig +short avalancheroofing.com
curl -sI https://avalancheroofing.com | head -1
gh api repos/peak11-clients/avalanche-roofing/pages \
  --jq '{cname,https_enforced,status}'
```

Peak 11 Media — internal technical SOP. System git is blocked; all pushes go through gh api via the deploy helper in /Business/PrimoSites/deploy/. Hosting is free on GitHub Pages; the only paid step is the client's domain, deferred until a deal closes.